

Khet Puzzle Solver

Ido Dukler

*Electrical & Computer Engineering
UCLA*

idukler@g.ucla.edu

Jeremy Euch

*Electrical & Computer Engineering
UCLA*

jeuchi@g.ucla.edu

Austin Motz

*Electrical & Computer Engineering
UCLA*

austinmotz@g.ucla.edu

Abstract—Khet, a laser-based strategy board game, combines elements of chess-like tactics with unique mechanics involving mirrors and beams. Despite its growing popularity, existing AI implementations for Khet struggle in deterministic end-game scenarios, particularly in identifying "win-in-N" solutions where optimal play guarantees a victory in a specified number of moves. This paper presents an optimal Khet Puzzle Solver designed to address this gap by leveraging an enhanced alpha-beta pruning algorithm. Our approach ensures computational efficiency while maintaining optimality in adversarial gameplay. Additionally, we developed a user-friendly interface that allows players to engage with puzzles interactively, mirroring the "daily puzzle" feature popularized by platforms like Chess.com. Through extensive benchmarking, we demonstrate the superiority of our solver against existing bots, which exhibit significant weaknesses in end-game problem-solving. By bridging the gap between theoretical AI optimization and practical gameplay, this project provides a robust platform for Khet enthusiasts and advances the state of AI in strategy board games.

I. PROJECT MOTIVATION

A. Board Games & AI

Board games have existed for thousands of years [1] and have been used for people's enjoyment. In the past few decades, advancements in computational ability and algorithm research have led to playable computational agents in these games.

In fact, one of the first AI algorithms was created to solve the game of checkers [3]. In recent years, AlphaGo has showcased the ability of AI to play well in even more complex games [4]. Alongside these advancements, we have seen an increase in demand for playable versions of these games [5]. Specifically, the recent popularity of chess and daily chess puzzles.

While chess and games alike have numerous online resources including playable versions, clubs, and strategy resources, our specific scope is focused on one of our personal favorite games, Khet. Khet is a two player game similar to chess, but with lasers. There are mirrored pieces with different characteristics which under some orientations reflect the laser. To win the game, the laser must hit the opponent's Pharaoh which is like checkmating the King in chess.

II. PROBLEM STATEMENT

A. Current Approaches

Currently, there exist some tools for the game Khet with playable AI's. To name a few, there is laser-chess.com [6], a

similar game inspired by Khet called Divine Light [7], and investigations into solving the game, notably Nijssen [1] and Zagury and Yaish [2].

Past work resulted in playable interfaces of the game and playable agents. However, current agents are built to work in the long term, but do not act optimally in end-game scenarios where there exists a "win-in-N" solution.

We aim to create a "win-in-N" Khet Puzzle Solver that is able to demonstrate that there exists a "win-in-N" given both players play optimally. In addition, we want to couple this solver with a playable user interface and hand-made puzzles to create a tool similar to the daily puzzle feature of Chess.com [8].

B. Our Solution

Our research explored the usage of an optimal algorithm for solving human-made Khet puzzles simulating end game scenarios. Through this investigation, we show the creation of an optimal "win-in-N" puzzle solver connected to a friendly, playable interface.

We decided to use an implementation of alpha-beta search, an extension of the two-player adversarial graph algorithm minimax, as we wanted explore the graph for an optimal solution to a win instead of using a learned approach for our agent. This guarantees optimality for both players in a computationally efficient time and offers a puzzle solving approach to the game.

III. GAME DYNAMICS & RULES

For the purpose of this investigation, we will not go into the formal rules of the game. At a high level, Khet is like chess, but with lasers. There are two players and 5 types of pieces: Pharaoh (like a king), Sphinx (shoots lasers), Scarab (two-sided mirror), Pyramid (one-sided mirror in two directions, killed otherwise), and an Anubis (accepts a laser in one direction, killed otherwise). The goal is to shoot lasers off of the angled surfaces to kill the opponent's team Pharaoh. For more details visit: Khet Official Rules [10].

IV. MATHEMATICAL FORMULATION

To define our problem, we will employ a graph representation of Khet. Each node will be a state of the Khet board such that it encapsulates the time history of inputs to our board, following the Markov Property and can be defined as a Markov

Decision Process (MDP). Each edge will represent a player making a move during their turn and evaluating the laser shot. So from one node to another, there could be a difference in move orientation in its board state, and a difference of the pieces of the board if any have been eliminated.

This setup allows for searching our graph to find the least number of moves to a state of the board which achieves the objective of Khet of eliminating the opponent player's Pharaoh.

A. State Definition

Each node in the graph is represented by a board including all of its pieces and their current orientations.

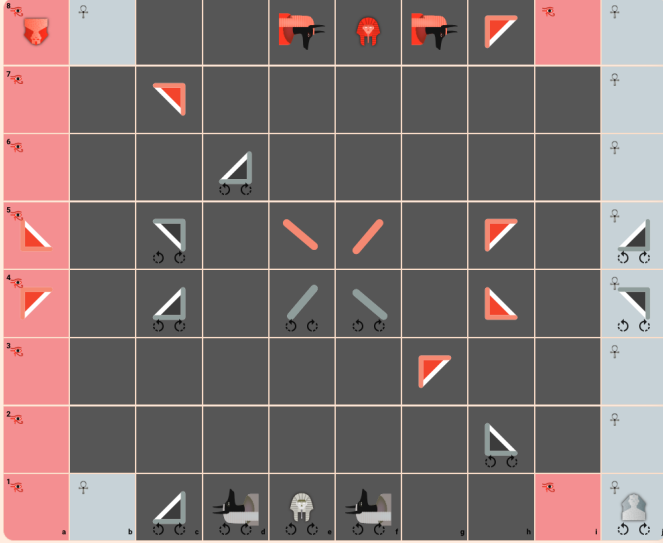


Fig. 1. Classic Khet Board Setup

Let S be the set of all possible states that the Khet board could take given its starting configuration seen in **Fig.1**. Let each possible board state $s_i \in S$ represent a board with all of its pieces and its orientations and is represented by an 8×10 matrix $M_i[8 \times 10]$.

To define each piece on some board location within M_i we encode each piece type, orientation, and color into the board representation.

$M_i[(x, y)] = (t, r, c)$
for $t \in \{\text{sphinx, pharaoh, pyramid, anubis, scarab, none}\}$
for $r \in \{\text{north, south, east, west, none}\}$
for $c \in \{\text{silver, red, none}\}$
for each (x, y) coordinate in M_i for $x \in X = \{1, \dots, 8\}$,
 $y \in Y = \{1, \dots, 10\}$.

Each piece at a board location can be represented in matrix notation $M_i[(x, y)] = (t, r, c)$ or by the tuple (x, y, r, t, c) . The board configuration in **Fig.2** can be described using the following notation:

$M_i[(8, 1)] = (\text{sphinx, north, silver})$

$M_i[(6, 6)] = (\text{pyramid, east, silver})$

$M_i[(8, 6)] = (\text{pyramid, east, silver})$

$M_i[(1, 8)] = (\text{sphinx, south, red})$

$M_i[(6, 8)] = (\text{pharaoh, north, red})$

$M_i[(i, j)] = (\text{none, none, none})$ for

$(i, j) \in \{X \times Y\} \setminus \{(8, 1), (6, 6), (8, 6), (1, 8), (6, 8)\}$

B. State Space Size

Khet is a complex game with many possible moves that can be made in any given turn. As a result, it has a very large number of possible outcomes defined in **Equation 1**. This helps understand the scope of the complexity of the game and allows for considerations on the approach to solving our proposed solution.

Defined in **Fig.1**, the classic Khet board starting configuration has 4 scarabs (2 silver, 2 red), 14 pyramids (7 silver, 7 red), 4 anubis (2 silver, 2 red), 2 pharaohs (1 silver, 1 red), and 2 sphinx, oriented and positioned as seen in the figure.

To find the number of different Khet boards that may result from an initial starting board, consider the order in which the different pieces can be placed or removed from the board.

There are 80 possible squares, for which 10 are silver squares and 10 are red squares. This leaves 70 possible squares for one color, silver for example, to place a piece to begin with. Now the sphinx is put at a fixed spot so that leaves 69 possible locations for the user to place the rest of the pieces. The evaluated expression below shows how the fixed position of the 2 sphinx's are placed, then the 1 pharaoh of one of the colors, followed by each of the 7 pyramids, each of the 2 scarabs, each of the 2 anubis's, for both colors and their respective orientations. This method follows Nijssen [1] and Zagury and Yaish [2] methods and assumptions to find this estimate for the state space approximation.

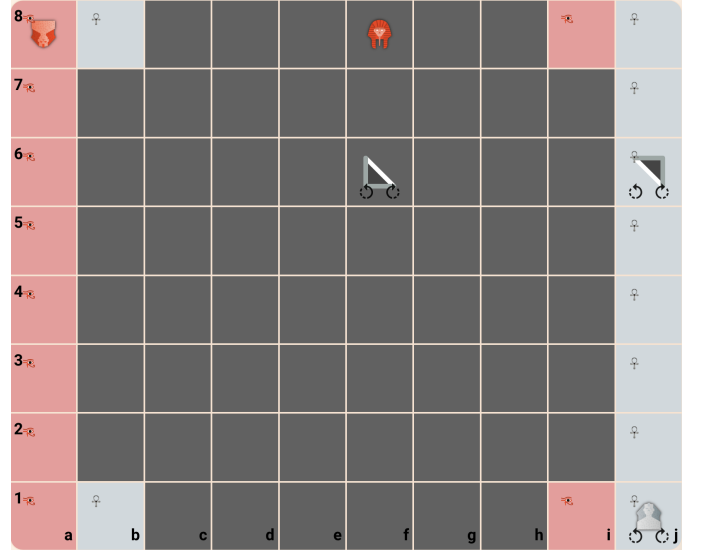


Fig. 2. Simple Board to Describe State

$$|S| = 2 \cdot 1 \cdot \binom{69}{1} \cdot 4^7 \cdot \binom{68}{7} \cdot 2^2 \cdot \binom{61}{2} \cdot 4^2 \cdot \binom{59}{2} \cdot 2 \cdot 1 \cdot \binom{56}{1} \\ \cdot 4^7 \cdot \binom{55}{7} \cdot 2^2 \cdot \binom{48}{2} \cdot 4^2 \cdot \binom{46}{2} \approx 1.222 \cdot 10^{46}$$

(Equation 1)

Khet State Space Size Approximation

C. Actions

Actions are a tuple $a = (m, z)$ where m is a piece and z is a move that piece makes. A_{color} describes the set of moves available to the player of that color in a state.

$$A_{color} = \{a = (m, z) \mid \text{Allowed}(M, a) \wedge m_c = color\}$$

The function $\text{Allowed}(M, a)$ will return a boolean value based on if action is allowed in the game rules. We use this for simplicity's sake as providing the exact mathematical formalization of this function would be lengthy and unnecessary to the understanding of our algorithm. Z describes the set of moves a piece can make. $z = (x, y, r)$ is an element of this set.

$$Z = \left\{ \begin{array}{l} \text{NORTH} = (0, 1, 0), \\ \text{NORTH_EAST} = (1, 1, 0), \\ \text{EAST} = (1, 0, 0), \\ \text{SOUTH_EAST} = (1, -1, 0), \\ \text{SOUTH} = (0, -1, 0), \\ \text{SOUTH_WEST} = (-1, -1, 0), \\ \text{WEST} = (-1, 0, 0), \\ \text{NORTH_WEST} = (-1, 1, 0), \\ \text{ROTATE_CW} = (0, 0, 1), \\ \text{ROTATE_CCW} = (0, 0, -1), \end{array} \right\}$$

The result of an action on a state, $R(s, a)$ consists of moving a piece, m_i on state s_i to obtain a new state, s_j . then evaluating the laser firing from the sphinx and removing the destroyed piece from M_j . The $\text{FireLaser}()$ function returns a set, $M_{destroyed}$, with at most one element, $m_{destroyed}$, if a piece was destroyed. Else, it returns the empty set, $M_{destroyed} = \{\}$.

$$m_j = (x_{m_i} + x_z, y_{m_i} + y_z, r_{m_i} + r_z, t_{m_i}, c_{m_i})$$

$$s_j = M_j = M_i \cup \{m_j\} \setminus \{m_i\}$$

$$M_{destroyed} = \text{FireLaser}(s_j, m_c)$$

$$R(s, a) = M_j \setminus M_{destroyed} \mid a \in A_{m_c}$$

A policy, π , will be defined as a list of allowed actions where silver moves first, then red, and so on.

$$\pi = (a_1, a_2, \dots, a_i)$$

$$\pi_{red} = (a_i \in \pi \mid i \% 2 = 0)$$

$$\pi_{silver} = (a_i \in \pi \mid i \% 2 = 1)$$

D. Graph Definition

To be able to represent a game we can create a graph, $G = (V, E)$ with vertices, V , of states achievable from a given initial state, s_i . A state, s_k is achievable when there exists a policy, π , that reaches the state from the initial state.

$$R(s, \pi) = R(R(R(s, a_1), a_2) \dots, a_n)$$

$$V = \{s_i\} \cup \{s_k \mid \exists \pi(s_k = R(s_i, \pi))\}$$

The edges of the graph, E , is a set containing tuples from one state, s_a , to another, s_b , such that a singular action, a_c , exists to connect them.

$$E = \{(s_a, s_b) \mid \exists a_c(s_b = R(s_a, a_c))\}$$

An important definition is depth, d , of a node, s_k , is the size of the policy, π_t , needed to traverse to that node from the root, s_i .

$$d = |\pi_t| \mid s_k = R(s_i, \pi_t)$$

This equation serves as a way to evaluate d on a state directly, but d will also be passed through functions later on for clarity. The value will be equivalent whenever it is used.

E. Utility

Let the Utility of a state be defined as the following:

$$U(s, d) = \begin{cases} \frac{100}{d} & (\dots, t = \text{Pharaoh}, c = \text{Red}) \notin M \\ \frac{-100}{d} & (\dots, t = \text{Pharaoh}, c = \text{Silver}) \notin M \\ 0 & \text{otherwise} \end{cases}$$

In a Puzzle solver this Utility works as an absolute metric, since the only thing that matters is a win state for either player and what node depth that win is at. If there is no win, then a state has zero value. A node having positive value guarantees that is a silver win, and a negative value guarantees it is a red win. It does not matter what moves led to the state.

F. Search Algorithms

1) Minimax Principle Definition

Khet is a two player zero-sum game, meaning each player's gain is the opposing player's loss [1]. If we think about the game from a perspective of your moves as the maximizing moves and your opponents moves as minimizing moves we can think about the minimax algorithm as the following:

A game state s_i representing the player's board before making a choice is the root node for the tree. Children of the node exist on the basis of the available edges (defined previously) each which have children based on their available edges. Minimax search evaluates all possible resulting game states $\{s_1, \dots, s_k\}$ where k is the number of possible states that exists starting at s_i within a horizon of d_{max} moves. Leaf nodes are the only states where utility is evaluated, and a leaf node exists only at the search depth or when a move is played which destroys a pharaoh. The algorithm defines the maximizer as the starting player and the minimizer as the opposing player. The value of a minimizer will be assigned as the value of the child with the least utility, and the value of a

maximizer will be assigned as the value of the child with the greatest utility.

$$MM(s, d, d_{max}, p) = \begin{cases} U(s, d+1) \text{ if } \{(\dots, t = \text{Pharaoh}, \dots), (\dots, t = \text{Pharaoh}, \dots)\} \not\subset M \vee d = d_{max} \\ \max_{a \in A(s)} MM(R(s, a), d+1, d_{max}, p = \text{Red}) \text{ if } p = \text{Silver} \\ \min_{a \in A(s)} MM(R(s, a), d+1, d_{max}, p = \text{Silver}) \text{ if } p = \text{Red} \end{cases}$$

The minimax function's value, v , simulates both players acting optimally. So, if there is a positive value, then the silver player is guaranteed to win in at most $\frac{100}{v}$ moves. If the value is zero, then there exists a sequence of moves that guarantees the silver player cannot win within the search depth d_{max} . Finally, if the value is negative, then the silver player is guaranteed to lose in at most $\frac{100}{v}$ moves.

2) Minimax Extension: Alpha-Beta Pruning

We decided to extend our Minimax algorithm using Alpha-Beta pruning due to computational limitations we encountered when experimenting with Minimax on puzzles.

In our analysis, we set up a simple win-in-2 puzzle with a board containing one red pharaoh, one silver pyramid, and both sphinxes.

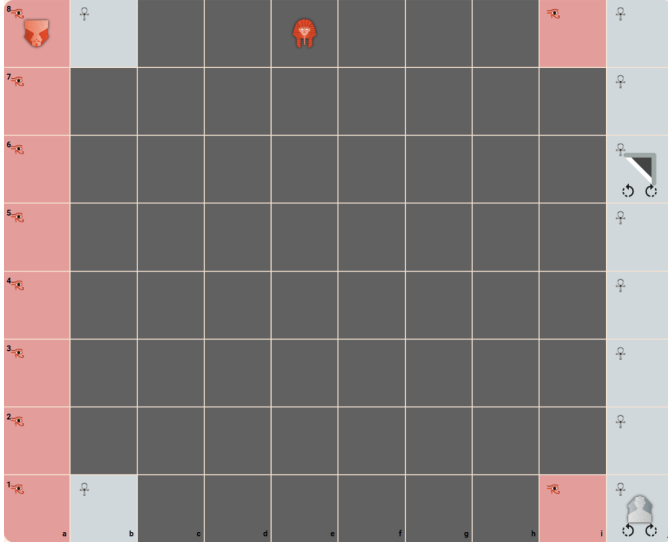


Fig. 3. Simple "Win-in-2" Board

TABLE I
NUMBERS REPRESENT THE NUMBER OF BOARD STATES VISITED TO FIND THE SOLUTION

Search Depth	Minimax Num. States	Alpha-Beta Num. States
4	3,376	367
6	215,705	3,884
8	84,000,000*	54,134

*Minimax did not terminate, this is a numerical estimate.

The minimax algorithm examines every available edge, exploring all possible resulting game states. Minimax explores states even if they are guaranteed to not be chosen as the resulting minimax value. As a result the computational limit increases at a rate of $(\text{average}_{\text{num moves}})^{\text{depth}}$.

G. How Alpha-Beta Works

Function alpha-beta ($node, d, isMax, \alpha, \beta$):

```

if  $d = d_{max}$  or  $node$  is terminal then
  | return  $U(node, d)$ 
end
if  $isMax$  then
  |  $v \leftarrow -\infty$ 
  | for each child of node do
  | |  $v \leftarrow \max(v, \text{alpha-beta}(child,$ 
  | |  $d + 1, \text{FALSE}, \alpha, \beta))$ 
  | end
  | if  $v \geq \beta$  then
  | | return  $v$ 
  | end
  |  $\alpha \leftarrow \max(\alpha, v)$ 
  | return  $v$ 
end
else
  |  $v \leftarrow +\infty$ 
  | for each child of node do
  | |  $v \leftarrow \min(v, \text{alpha-beta}(child,$ 
  | |  $d + 1, \text{TRUE}, \alpha, \beta))$ 
  | end
  | if  $v \leq \alpha$  then
  | | return  $v$ 
  | end
  |  $\beta \leftarrow \min(\beta, v)$ 
  | return  $v$ 
end

```

Fig. 4. Alpha-Beta Search Algorithm (derived from Russel and Norvig [9])

Alpha-beta pruning works by exploring the minimax game tree while keeping track of two values α and β . α keeps track of the maximizer's worst guaranteed value and β keeps track of the minimizer's best possible value. Their initialization is $\alpha = -\infty, \beta = \infty$ to demonstrate the worst possible case for each.

Similar to minimax, the exploration begins at a maximizer root node, $node$, and begins investigating potential maximizer decisions. When investigating the first move, you recursively explore every possible minimizer decision as well. This exploration continues until a termination node is reached (either at a minimizer or maximizer step) or the depth of the search has been reached. At this point the utility value v of the state is assigned. This v "bubbles up" the tree and updates each subsequent node's α and β respectively [9].

Our game tree contains nodes with v either indicating there is no win in the future horizon $v = 0$, there is a win in the future $v = \frac{100}{q}$ for some win at some depth q , or a future loss $v = \frac{-100}{r}$ for some win at some depth r , for $q, r \leq d_{max}$.

If $v \leq \alpha$ when searching minimizer decisions, this means a minimizer optimal utility v is lesser than the greatest previously guaranteed lowest utility minimizer response α . Meaning, one of the previously explored maximizer decisions led to a minimizer optimal utility that is greater than the current. From the maximizer's perspective, there is no need to continue searching this set of minimizer responses, as the minimizer step of the algorithm will choose v or a lower explored utility as its new optimal minimizer response to the maximizer decision and update $\beta = v$. As the maximizer, you can safely prune this branch since you have a guaranteed better alternative path indicated by α .

If $v \geq \beta$ when searching maximizer decisions, this means the value of the current maximizer node v is better than the best value the minimizer can guarantee itself β . This β exists at some higher level in the tree outside of the maximizer's current subtree where v was found. This means that the minimizer would never choose this path to explore as it knows it had already found a better one earlier in its search. As the minimizer, you can safely prune this subtree as this maximizer decision would never be part of the final path.

For an intuitive worked out example of alpha-beta search, refer to pg.168 of Russel and Norvig [9].

H. Doubts

One consideration of pruning in an optimal algorithm is if pruning the tree will affect the algorithm's optimality if non-optimal moves are made by the adversary.

We propose a proof by induction to show that the search space of the minimax game tree will result in no different of a practical search tree than the alpha-beta search approach and will result in an optimal policy assumed an optimal adversary or better.

I. Alpha-Beta Proof

To prove alpha-beta pruning will correctly find a guaranteed win if one exists within the game tree up to depth d_{max} we will use a proof by induction.

Base Cases:

- **Maximizer Reaching Termination**

Under Minimax search, each maximizer action is explored. After exploring all possible values of each of the nodes, the action that resulted in the max value node is taken as the Minimax policy for maximizer step.

Under Alpha-Beta, we would keep track of the minimum guaranteed reward α and the maximum possible reward β . If $\alpha \geq \beta$, the exploration has found the highest possible value the maximizer can achieve for this turn. As a result, the exploration terminates and the rest of the tree is pruned.

Since the maximum possible value is $\frac{100}{d+1}$, we can set the max value of β to be equal to this instead of ∞ , and the value of $\alpha = \frac{-100}{d+1}$. In the appendix, **Fig.7** has figures (1a)-(1c) shows the maximizer finding a child value equal to $\frac{100}{d+1}$, which makes the final value of the

node as follows.

$$value = [\frac{100}{d+1}, \frac{100}{d+1}] = \frac{100}{d+1}$$

The rest of the tree can be pruned because given our heuristic definition we know there cannot possibly be a node of greater value. They will all be at most equivalent.

- **Minimizer Reaching Termination**

Under Minimax search, each minimizer action is explored. After exploring all possible values of each of the nodes, the action that resulted in the minimum value node is taken as the Minimax policy for the minimizer step.

Under Alpha-Beta, we would keep track of the minimum guaranteed reward α and the maximum possible reward β . We are looking to find the actions that forces the maximizer to have the lowest possible value. If we come across a $value \leq \alpha$ or we come across a terminating value, i.e. a red win with $value = \frac{-100}{d}$, we do not need to explore further. This is because either red won, or there is a previous minimizer decisions leading to a worst case maximizer value α that is better than the minimum optimal value for the minimizer for this step.

Since the minimum possible value is $\frac{-100}{d}$, once Minimax explores all options, the first action which produces this minimum possible value for the given turn will be chosen. This means that Minimax did not need to evaluate the rest of the minimizer decisions to choose the Minimax action. This is illustrated in figures (2a)-(2b).

- **Maximizer Reaching Termination with No Winning Moves**

With minimax search, this base case would be equivalent, but with Alpha-Beta there will be no pruned nodes. In this case Alpha-Beta will evaluate all children and select the highest value node, whether that be a node which leads to no win for either player, or one that leads to a win for the red player. Figures (3a)-(3c) illustrate this.

- **Minimizer Reaching Termination with no way to escape a Silver win**

Again, the minimax search in this case will be equivalent to the minimizer reaching termination case described before, as both cases minimax will evaluate all nodes. Alpha-Beta pruning will also evaluate all children and verify that all have a value of $\frac{100}{d+1}$, signifying a silver win.

Now, assume that Alpha-Beta pruning will correctly evaluate a game tree of depth k . We will now show $P(k+1)$ is correct by evaluating the alpha-beta graph at depth $k+1$ and show that it correctly evaluates in all cases.

Figures (5a)-(5c) show a minimizer correctly evaluating the game tree and pruning unneeded nodes because there existed a red win as a child. Figures (6a)-(6c) shows a minimizer

correctly evaluating maximizers who all obtain a silver win in some time horizon. All children are evaluated and the minimizer correctly selects the node with the least value, ie. the furthest possible silver win. Finally, figures (7a)-(7c) show the Alpha-Beta algorithm finding a win-in-N by evaluating all children's values to ensure there is not a shorter possible path to a win.

Therefore, the following statement holds. If the evaluation of the alpha-beta algorithm on a state, s_i is greater than 0. Then, for all actions red can take, π_{red} , in the set of all sequences of actions allowed from the starting state, Π_i , there exists an optimal sequence of actions from silver, π_{silver}^* , where silver will win and the total length of the policy is less than or equal to one hundred divided by the value of the initial state.

$$m_{RP} = (... , t = Pharaoh, c = Red)$$

$$S_{puzzles} = \{s_i \in S \mid \text{alphabeta}(s_i, 0, -\infty, \infty, True) = value > 0\}$$

$$\forall s_i \in S_{puzzles} (\forall \pi_{red} \in \Pi_i (\exists \pi_{silver}^* \in \Pi_i (m_{RP} \notin R(s_i, \pi) \wedge |\pi| \leq \frac{100}{value})))$$

V. IMPLEMENTATION

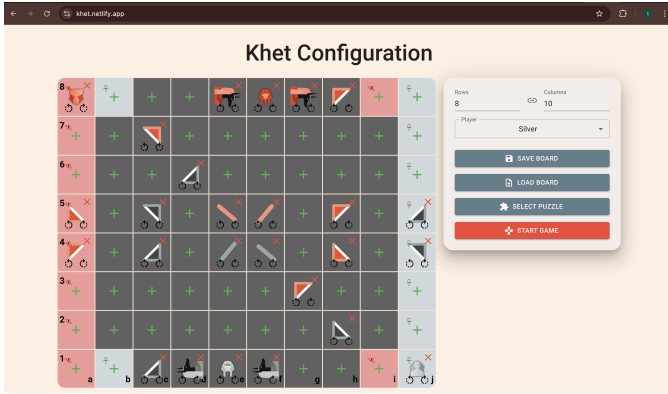


Fig. 5. Khet Website

The frontend as seen in **Fig.5** was built from the ground up to serve as both a tool for playing the game of Khet and a platform for creating puzzles by configuring the board with different pieces. During development, we identified significant limitations in existing repositories, which influenced our decision to create a custom solution. The KhetAI repository, while offering strong AI implementations, lacks a playable interface and focuses exclusively on bot-vs-bot matches. Additionally, its codebase is poorly documented, making modifications difficult. Similarly, the repository from laser-chess.com, although equipped with a more functional UI, relies on outdated packages and lacks backend integration, limiting its usability. To

address these issues, our frontend provides dual game modes: users can play as either the Red or Silver player. Playing as Red allows users to defend against Silver's AI, showcasing the algorithm's integrity as Silver consistently finds the optimal solution and achieves checkmate regardless of the user's moves. Conversely, playing as Silver lets users take the role of the attacker, attempting to solve the puzzle while the opponent minimizes their chances of success. In addition to gameplay, the frontend enables users to configure the board with pieces for custom puzzle creation. To make this tool accessible, we deployed the frontend using Netlify for easy demos; however, due to RAM limitations on pythonanywhere.com, users must clone the repository and run the backend locally to activate the AI functionality. This custom frontend not only resolves the shortcomings of existing repositories but also provides a comprehensive platform for gameplay, puzzle creation, and AI exploration.

VI. RESULTS AGAINST EXISTING AI

The performance of our AI puzzle solver was benchmarked against several bots from the KhetAI [2] repository, including **best.json**, **SDK Default**, **HeuristicBot**, and **UserBot**, as well as the heuristic-based solver available on **laser-chess.com** [6]. These systems represent the best-known implementations for Khet AI but consistently underperformed in end-game scenarios requiring "win-in-N" solutions. We conducted 5 trials for each bot against our custom made Hand of Anubis puzzle which is a win-in-3 for silver.

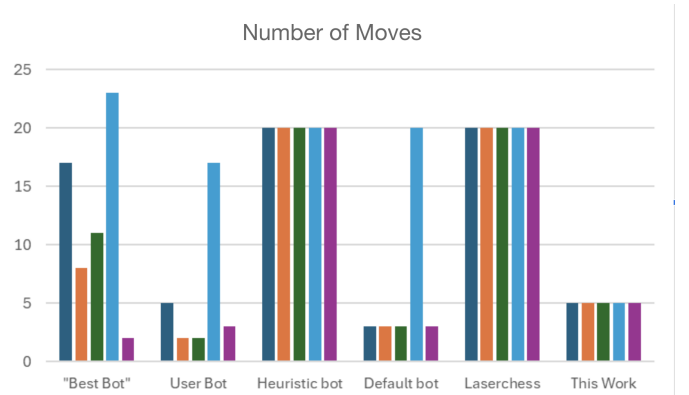


Fig. 6. Results of our solution against current state-of-the-art

A. Limitations of the Bots

- **best.json**: Touted as the most optimized configuration in the repository, exhibited significant issues in solving deterministic end-game puzzles. It frequently blundered in critical positions and self-destructive moves, such as killing its own Pharaoh, and entered repetitive move loops, failing to progress toward a solution.
- **UserBot**: This bot showed similar limitations to best.json, including a high tendency for self-kills and an inability to find optimal moves in end-game scenarios. Although

slightly more adaptable in earlier game stages, it faltered in deterministic puzzles that require precise "win-in-N" solutions.

- **HeuristicBot:** It displayed purely random behavior beyond a few early-game tactics. It lacked any meaningful strategy for mid-to-late game situations, let alone specialized end-game problem-solving.
- **SDK Default:** This bot serves as a baseline implementation in the KhetAI repository. As expected, its moves were largely random, with no apparent ability to handle complex positions or end-game puzzles.
- **laser-chess.com:** Their bot employs a heuristic strategy, but its evaluations are overly simplistic and fail to address the nuances of "win-in-N" puzzles. It often prioritizes suboptimal moves and struggles to handle deeper search depths, making it unsuitable for direct comparison to our approach.

B. Our Approach

Our AI solver is designed specifically to tackle deterministic and depth-limited puzzles. By narrowing our focus to targeted end-game problem-solving, we avoid the shortcomings observed in the KhetAI bots and the laser-chess.com solver shown in **Fig.6**. Key advantages of our solver include consistency which is to say it's optimal and deterministic. It is fine-tuned for end-game scenarios, achieving perfect accuracy where other bots fail. In all test cases, our solver demonstrated flawless performance, consistently outperforming KhetAI bots and the laser-chess.com solver.

C. Decision to Limit Further Testing

Given the observed weaknesses in the KhetAI bots and laser-chess.com solver for end-game "win-in-N" scenarios, we decided to forego additional testing. While these systems represent the best publicly available benchmarks, their designs prioritize full-game strategies and lack the depth and precision required for solving targeted end-game puzzles. As such, their inclusion provides necessary context but does not detract from the clear superiority of our solver in its specialized domain.

VII. CONCLUSIONS

Our findings highlight the distinct advantages of our proposed solver, which not only outperforms existing AI implementations in addressing deterministic end-game configurations in Khet, but also establishes a new benchmark for accuracy and reliability in "win-in-N" puzzles. In contrast to broader, less targeted solutions, our method thrives within its specialized domain, demonstrating a level of analytical precision and consistency previously unseen in automated Khet gameplay.

Beyond the technical refinements, the introduction of a user-friendly, puzzle-style interface further amplifies the impact and reach of our solver. By allowing enthusiasts to engage with intricately crafted end-game challenges in an accessible, interactive format, we provide an opportunity for players to

deepen their strategic understanding and appreciation of the game. As a result, our work not only contributes to the ongoing refinement of computational Khet strategies but also enriches the player experience, offering both a practical tool for skill development and a gateway to deeper exploration of advanced end-game dynamics.

VIII. ACKNOWLEDGEMENTS

In our project, we utilized generative AI tools to aid us build the UI and help with code some completion on the backend. We specifically used ChatGPT and GitHub Copilot. Thank you Professor Ankur Mehta for guiding us during the development of our project.

REFERENCES

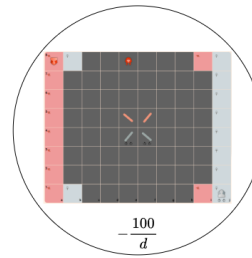
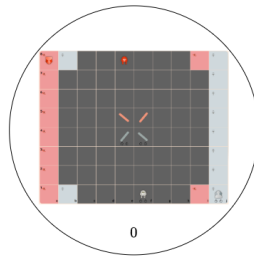
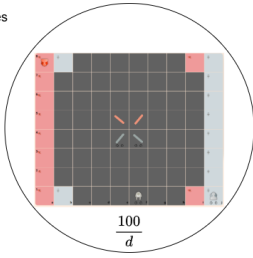
- [1] Nijssen, J. P. A. M., and J. W. H. M. Uiterwijk. "Using intelligent search techniques to play the game Khet." Maastricht Univ., Netherlands, Netherlands, Tech. Rep (2009).
- [2] Aviv Yaish. "KhetAI" GitHub, 2018, github.com/AvivYaish/KhetAI/blob/master/paper.pdf. Accessed 13 Dec. 2024.
- [3] IBM. "The Games That Helped AI Evolve | IBM." [www.ibm.com, www.ibm.com/history/early-games](https://www.ibm.com/history/early-games).
- [4] "AlphaGo by DeepMind: The AI That Mastered Go." Perplexity AI, 2016, www.perplexity.ai/page/alphago-by-deepmind-the-ai-that-zpNCBbdIQ4eAcjGHwvq4dA. Accessed 13 Dec. 2024.
- [5] Keener, Greg. "Chess Is Booming." The New York Times, 17 June 2022, www.nytimes.com/2022/06/17/crosswords/chess/chess-is-booming.html.
- [6] "Laser Chess." Laser-Chess.com, 2024, laser-chess.com/. Accessed 13 Dec. 2024.
- [7] Yves Ruosch. "Divine-Light." Divine-Light.ch, 2024, divine-light.ch/home. Accessed 13 Dec. 2024.
- [8] "Daily Chess Puzzles - Your Daily Chess Challenge." Chess.com, www.chess.com/daily-chess-puzzle.
- [9] Russel, Stuart, and Peter Norvig. Artificial Intelligence: A Modern Approach. 4th ed., Prentice Hall, 2020.
- [10] "Khet Rules: Official Game Rules." Officialgamerules, www.officialgamerules.org/khet. Accessed 13 Dec. 2024.

IX. APPENDIX

A. External Links

To access the code repository including both our frontend, backend, and our Netlify app, access our GitHub here: [Khet](#).

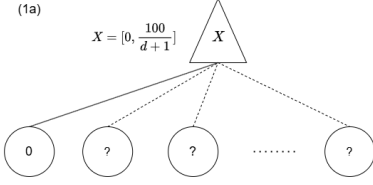
Termination States



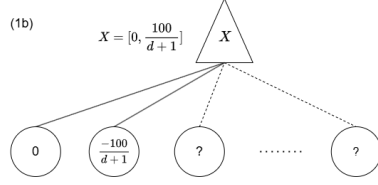
d

Maximizer Reaching Termination - Finding Winning move for player

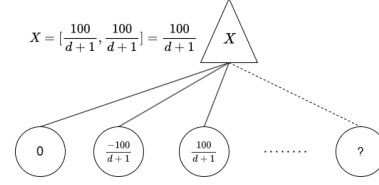
(1a)



(1b)



(1c)

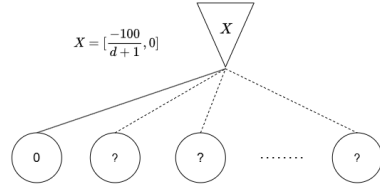


d

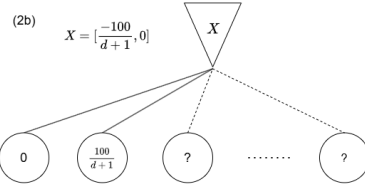
$d+1$

Minimizer Reaching Termination - Finding winning move for opponent

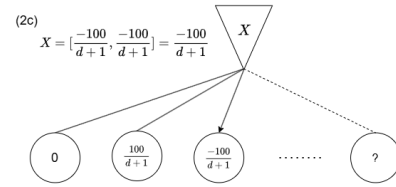
(2a)



(2b)



(2c)

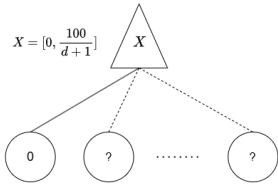


d

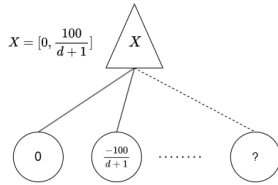
$d+1$

Maximizer Reaching Termination with no winning moves for player

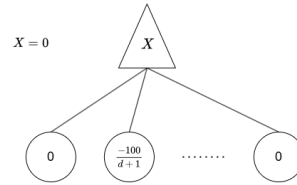
(3a)



(3b)



(3c)

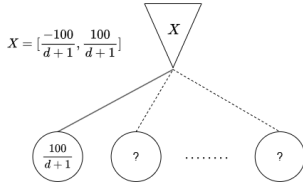


d

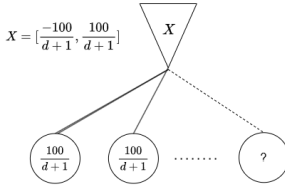
$d+1$

Minimizer Reaching Termination - with no way to escape a player win

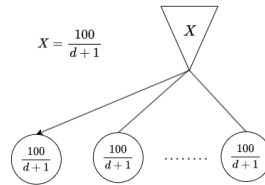
(4a)



(4b)



(4c)

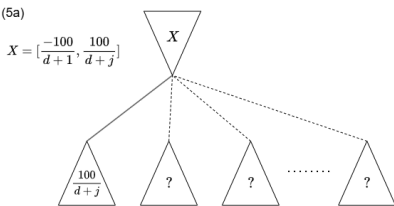


d

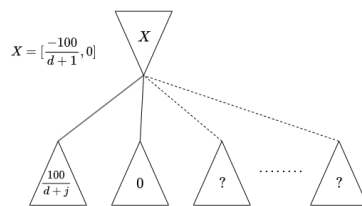
$d+1$

Minimizer Evaluating a Maximizer - I.E. Escaping a Win

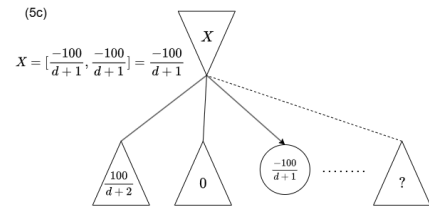
(5a)



(5b)



(5c)

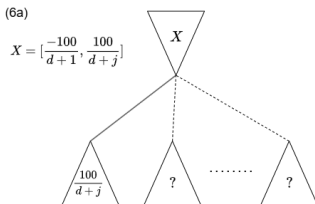


d

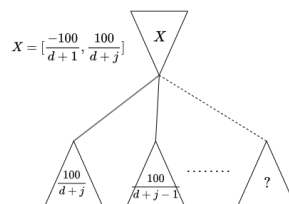
$d+1$

Minimizer Evaluating a Maximizer where termination is in j steps - I.E. Loss after j steps

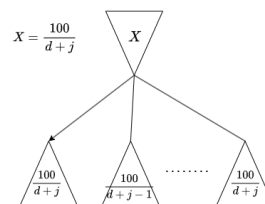
(6a)



(6b)



(6c)

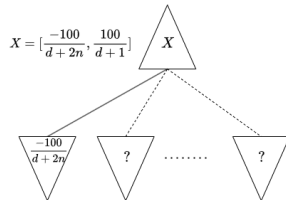


d

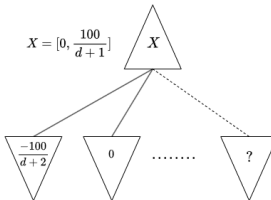
$d+1$

Win In n Player Moves

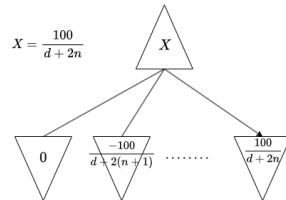
(7a)



(7b)



(7c)



d

$d+1$

Fig. 7. Figures used for the Alpha-Beta Pruning Proof